LibraryDeskSense: An IoT System for Occupancy, Noise, and Light Monitoring of Library Desks

Dept. of Computer Science and Engineering
University of Bologna, Bologna, Italy
Sina Mohebbi - sina.mohebbi@studio.unibo.it
Yousef Barjas - yousef.barjas@studio.unibo.it

Abstract-LibraryDeskSense is a single-desk IoT prototype that reports whether a library study desk is occupied, how loud it is, and how well lit it is, without ever recording audio or identifying who is using it. An ESP32 node reads an HC-SR04 ultrasonic sensor, a VEML7700 light sensor, and a MAX4466 microphone, and sends periodic telemetry over HTTP or CoAP and asynchronous events and configuration over MQTT to a Python proxy that stores everything in InfluxDB. A Grafana dashboard, a set of analytics scripts, and a Telegram bot all read from the same database independently. The system also switches between HTTP and CoAP based on measured latency, recommends the quietest and best-lit study hours, and pushes real-time alerts over Telegram. We validated the full pipeline on real hardware over a three-hour deployment with deliberately varied conditions, collecting 6154 telemetry samples across 5 occupancy sessions. CoAP outperformed HTTP on every latency statistic and used less than half the estimated bytes per message. For forecasting, noise is modeled with ARIMA while light uses a persistence model; both are evaluated with MAE and written back to InfluxDB for Grafana.

Keywords—*Internet of Things, ESP32, occupancy sensing, CoAP, MQTT, time-series forecasting, Grafana, InfluxDB.*

I. INTRODUCTION

Shared study spaces are a limited resource in most university libraries. Students routinely walk between floors looking for a desk that is actually free, quiet enough to concentrate, and lit well enough to read comfortably, while library staff have essentially no quantitative picture of how those desks are used throughout the day. At the same time, monitoring a personal workspace raises an obvious privacy concern: nobody wants a microphone recording what is said at their desk. LibraryDeskSense is our answer to both problems. It is a single-desk IoT prototype that reports whether a desk is occupied, how loud its surroundings are, and how well lit it is, without ever identifying who is sitting there or recording anything they say.

LibraryDeskSense was built as a complete end-to-end IoT system rather than only a small sensor demo. The final prototype combines an ESP32 sensing node, FreeRTOS firmware, HTTP and CoAP telemetry, MQTT events and runtime configuration, InfluxDB storage, Grafana visualization, Python analytics, and Telegram notifications. This makes it possible to follow one desk from the physical sensor reading all the way to live dashboards, alerts, forecast values, and evaluation results.

The rest of this paper is organized as follows. Section II describes the overall architecture of the system, from the firmware task layout to the data pipeline. Section III describes how each part was actually implemented, including the debounce and hysteresis logic that make the sensor readings usable and the adaptive protocol selection logic. Section IV reports the results collected from a real deployment session, including forecasting accuracy and the HTTP-vs-CoAP latency and overhead comparison.

II. PROJECT ARCHITECTURE

A. System Overview

LibraryDeskSense is organized as a pipeline with four stages. An ESP32 board mounted on the desk reads the three sensors and turns them into two kinds of messages: periodic telemetry samples and one-off events. Telemetry is pushed to a Python proxy over HTTP or CoAP; events, together with runtime configuration changes, travel over MQTT through a local Mosquitto broker. The proxy is the only component that talks to InfluxDB, so it is the single point where all three transports converge into one time-series database. From there, three independent consumers read the same data: a Grafana dashboard for live visualization, a set of Python analytics scripts for trend analysis and ARIMA forecasting, and a Telegram bot for on-demand queries and push alerts. No component other than the proxy writes to InfluxDB, which keeps the storage layer simple and consistent. Fig. 1 summarizes this data flow.

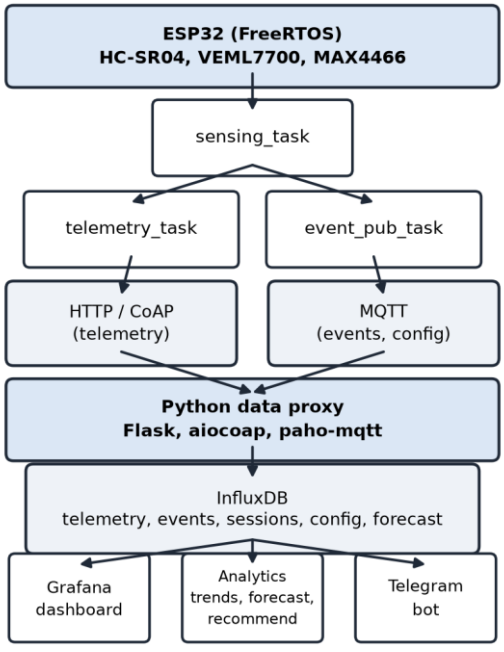

Fig. 1. LibraryDeskSense system architecture and data flow.

B. Firmware Task Architecture

The firmware is split into three FreeRTOS tasks instead of one large loop, specifically so that the two network sends—which can each block for up to four seconds if the proxy is slow or unreachable—never delay the next sensor sample. The sensing_task task reads the ultrasonic, light and sound sensors on a fixed period, runs the occupancy/noise/light state machines described in Section III, and pushes results onto two FreeRTOS queues. The telemetry_task task blocks on the telemetry queue and sends each sample over HTTP or CoAP.

The event_pub_task task blocks on the event queue and publishes each event over MQTT. Because sending is isolated in its own task, a slow or dropped network connection only delays telemetry delivery, not sensing.

The three tasks communicate through a small, fixed set of shared objects: two bounded queues (one for telemetry samples, one for events), a single configuration struct protected by a mutex, and a FreeRTOS event group with one bit for Wi-Fi connectivity and one for MQTT connectivity. telemetry_task and event_pub_task both wait on the relevant connectivity bit before sending, so neither task needs its own reconnect logic; they simply resume automatically once the Wi-Fi driver or the MQTT client report that the link is back. If a queue is full because the network is down for a long time, new samples are dropped rather than blocking the sensing loop, and the drop is logged.

C. Communication Architecture

Telemetry uses HTTP or CoAP, selected by a comm_mode setting that can be fixed to either protocol or left on auto. In auto mode the firmware keeps an exponentially weighted moving average of the round-trip latency of each protocol and sends each new sample over whichever one currently looks faster, while occasionally probing the other protocol so that the estimate does not go stale. Events (desk_occupied, desk_released, high_noise_event, poor_lighting_event) and runtime configuration changes (sampling_ms, noise_thr, light_thr, occupancy_distance_cm, occupancy_timeout_ms, comm_mode) both use MQTT, since they are asynchronous and low-rate rather than periodic.

D. Data and Software Architecture

InfluxDB stores six measurements: telemetry (one point per sensing cycle), events, occupancy_sessions (written once a session ends, with its duration), config_changes (an audit trail of every applied runtime configuration), and forecast and forecast_metrics (written by the forecasting module). The proxy is a single Python process that runs a Flask HTTP server and a paho-mqtt client each on their own thread, and an aiocoap server on the main asyncio event loop, so it can accept all three transports concurrently without blocking one on another. Grafana, the analytics scripts, and the Telegram bot all read InfluxDB independently and do not talk to each other or to the proxy directly.

III. Project Implementation

A. Hardware and Sensing

Occupancy is inferred from an HC-SR04 ultrasonic sensor pointed at the seat: the firmware times the echo pulse, converts it to a distance, discards anything below 2 cm or above 400 cm as an invalid reading, and takes the median of three consecutive samples to filter out single-pulse glitches. Light is read from a VEML7700 ambient-light sensor over I2C, which returns lux directly. Sound is the sensor with the tightest privacy constraint: a MAX4466 electret microphone is sampled 256 times through the ADC, and the firmware reduces those samples to one RMS deviation value, which is used only as a sound-intensity score. The 256 raw samples exist only inside one function call and are discarded immediately afterward; no audio waveform is ever queued, logged, stored, or sent over the network. Fig. 2 shows the assembled sensor wiring.

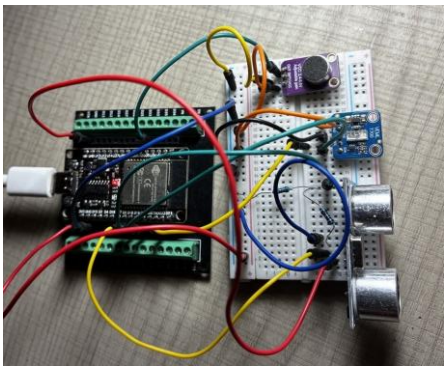

Fig. 2. The ESP32 board wired to the HC-SR04, VEML7700, and MAX4466 sensors.

B. Occupancy and Event Detection

A raw distance reading below threshold is not, by itself, enough to declare the desk occupied: a person shifting in their chair or leaning out of range for a moment would otherwise generate false desk_released events. We use a small debounce state machine instead: entering the occupied state requires three consecutive present readings, and leaving it requires an occupancy_timeout_ms period (configurable at runtime, 15 s by default) with no present reading at all. High-noise detection is intentionally more immediate: one reading above the configured threshold sends one high_noise_event, and the event is not allowed to re-arm until the noise falls below 80% of that threshold. Poor-lighting detection is slower and uses two consecutive low readings plus a clear margin, because light changes near the threshold can otherwise create repeated alerts.

One subtlety we found and fixed during testing is worth mentioning: the HC-SR04 has a physical blind zone below roughly 2 cm, so an object placed very close to the sensor produces no valid echo at all rather than a small distance value. The first version of the debounce logic treated an invalid reading exactly like a confirmed absent reading, which meant a burst of invalid echoes while someone leaned in close to the sensor could eventually time out an active session even though the desk was still occupied. The fix was to simply skip the debounce update entirely whenever the distance reading is invalid, so the timeout clock only advances on genuine evidence of absence.

C. Adaptive Protocol Selection

The auto communication mode keeps two floating-point latency estimates, one per protocol, updated after every successful send with the exponentially weighted moving average new_average = 0.3 * latest_latency + 0.7 * old_average. Once both protocols have at least one measurement, the firmware normally sends over whichever one has the lower average latency, but every tenth sample it deliberately probes the other protocol instead, so that a protocol which happens to become faster later is still noticed. A failed send is penalized with a fixed 4000 ms latency estimate for that protocol, which makes the firmware avoid it until it recovers, and in auto mode a failed send is immediately retried once on the other protocol before the sample is given up on.

D. Data Proxy and Storage

The proxy exposes a POST /telemetry endpoint for HTTP, a matching CoAP resource at /telemetry, and a POST /config endpoint used by a small command-line tool to push runtime

configuration changes. A config change is published to the desk's MQTT config topic and, because the proxy also subscribes to that same topic, it is recorded as a config_changes point the moment it is published, giving a complete audit trail of what was applied and when. Every telemetry point is tagged with which transport delivered it, which is what lets the adaptive protocol selection behavior discussed in Section IV-D be analyzed after the fact, without any separate logging on the firmware side.

E. Analytics

Three Python scripts analyze the stored data. trends.py computes desk utilisation, average session duration, mean noise and light, the busiest hour, the quietest hour, and a count of every event type observed, and saves the result as a JSON summary. forecast.py resamples noise and light into one-minute buckets, keeps only the most recent continuous block of data, evaluates the model on a chronological test window using MAE, and writes one forecast value for the next 15-minute window. Noise is forecast with ARIMA(0,1,1), while light uses a persistence model because the collected light signal is mostly stable between sudden lamp changes. recommend.py scores each 15-minute time-of-day slot (configurable via --slot-minutes) by availability, quietness, and light quality to suggest the best study periods.

F. Grafana Dashboard

The Grafana dashboard is the main live view of the system. It shows current desk occupancy, average session duration, 24-hour utilisation, current light level, noise level over time, light intensity over time, high-noise and poor-lighting events, and forecasting accuracy. The forecast accuracy table shows the signal, model, forecast value, and MAE for each signal; the forecast is reported there rather than overlaid on the time-series charts, which show the measured signals only. All panels are scoped to a desk template variable, so the same dashboard can be reused for another desk id.

G. Telegram Bot

The bot supports three commands over long-polling: /start prints a short help message, /status reports the desk's most recent occupancy, session length, noise, and light reading, and /stats reports 24-hour averages. Independently of those commands, a background thread subscribes to the desk's MQTT event topic and pushes a message the moment a high_noise_event or poor_lighting_event is published, so a user can be notified without watching Grafana.

IV. Results

A. Test Methodology

We deployed the ESP32 node on a real desk and ran the complete pipeline-firmware, proxy, InfluxDB, Grafana, and the Telegram bot-continuously for a session of just over three hours. Rather than leaving the desk in one static condition, we deliberately varied it: the desk was left empty for short periods to generate genuine desk_released events, the desk lamp was switched off briefly on more than one occasion to trigger poor_lighting_event, and several short bursts of conversation-level noise were produced near the microphone to trigger high_noise_event. This gave the dataset enough variation to test the event logic, dashboards, analytics scripts, and protocol selection under realistic conditions.

B. Telemetry and Event Statistics

Over the session, the proxy received 6154 telemetry samples, delivered over a mix of HTTP and CoAP chosen by the firmware's adaptive protocol selection, giving 66.2% overall desk utilisation across 5 completed occupancy sessions with an average duration of 704 s. The busiest hour of the day was 21:00 and the quietest was 18:00; to stop a short partial hour at the edge of the collection window from winning by chance, we consider only hours with at least 20 samples and break ties by sample count, so the fully-sampled 21:00 (n=1355) is ranked ahead of a 78-sample 17:00. Table I shows the number of each event type observed; note that desk_occupied (8) exceeds desk_released (5) because three occupancy sessions were still open when recording ended.

TABLE I. Event Counts Over the Test Session

Event type	Count
desk_occupied	8
desk_released	5
high_noise_event	20
poor_lighting_event	5

C. Forecasting Results

Table II reports the latest forecasting results for noise and light, evaluated on a chronological held-out test window using mean absolute error (MAE). The table also includes the single forecast value written for the next 15-minute window. Noise uses ARIMA(0,1,1); light uses a persistence model because the lux readings are usually stable unless the lamp or room lighting changes suddenly.

TABLE II. Forecast Accuracy (MAE)

Metric	Model	Forecast value	MAE
Noise	ARIMA(0,1,1)	35.46	2.07
Light	Persistence	257.9 lux	1.08

The forecasting result is intentionally simple and explainable. Noise has enough short-term variation for ARIMA to be useful as a demonstration model, while light is better represented by persistence because the reading usually remains nearly constant until a lamp or room-light change occurs. Using MAE keeps the evaluation easy to interpret: it reports the average absolute error in the same units as the original signal. Against a naive persistence baseline, ARIMA reduced the noise-forecast error by about 23% (MAE 2.07 versus 2.70), whereas for light ARIMA offered no improvement over persistence, confirming persistence as the appropriate model for the near-constant lux signal. We tried several standard models for each signal and kept the ones that worked best on our data. This deployment was realistic but shorter and less varied than a full day in a working library, so with a longer collection period and thresholds configured for a specific desk we would expect these models to forecast the conditions very well in practice.

D. HTTP vs. CoAP Evaluation

We benchmarked both protocols directly against the running proxy using 100 identical telemetry payloads per protocol, sent from a separate benchmark desk identifier so the measurements would not mix with the real desk's data. Table III reports round-trip latency and the estimated per-message overhead, where the header estimate accounts for typical HTTP request headers versus CoAP's compact binary header and excludes IP/TCP/UDP framing.

TABLE III. HTTP VS. COAP LATENCY AND OVERHEAD (N = 100)

Protocol	Mean (ms)	Median (ms)	p95 (ms)	Bytes/msg
HTTP	59.6	59.9	72.2	277
CoAP	47.1	46.9	49.4	113

CoAP was faster on every statistic and needed less than half the estimated bytes per message, mainly because it avoids HTTP's text-based headers and uses UDP instead of a TCP handshake. This matches what we observed from the firmware's adaptive protocol selection during the test session: with both protocols reachable and no fixed comm_mode forced, the EWMA latency tracker consistently favored CoAP once it had a few measurements of both, which is exactly the behavior Section III-C was designed to produce.

E. Quietness Recommendation

Table IV lists the top three 15-minute time-of-day slots identified by the recommendation module, each of which had at least 20 samples and at least 10% availability. The 18:30–18:45 slot scored highest — fully free during the session, quiet (90th-percentile noise 30), and adequately lit — ahead of two early-evening slots around 20:00–20:30.

TABLE IV. TOP-3 RECOMMENDED STUDY SLOTS (15 MIN)

Time slot	Score	Free %	Noise p90	Median lux
18:30–18:45	0.91	100.0%	30.0	176.8
20:15–20:30	0.70	97.7%	52.0	187.8
20:00–20:15	0.67	100.0%	54.0	171.5

F. Discussion

The results show that the prototype works as a complete IoT pipeline: both telemetry protocols were exercised and compared quantitatively, forecasting was evaluated with MAE against a meaningful baseline rather than reported in isolation, and all four MQTT event types were observed with real hardware. The main limitation of this evaluation is scale: one desk and a bit over three hours of data is enough to demonstrate that every part of the pipeline works end to end and to produce realistic example analytics, but it is not enough to claim long-term behavioural patterns for a real library. Two implementation caveats also bound the results. First, the ESP32 has no SNTP time synchronisation, so each sample carries only a boot-relative timestamp, which the proxy ignores in favour of its own server-side arrival time; this is sufficient for a single node but would need real device clocks for a multi-node deployment. Second, in AUTO mode a telemetry sample can in rare cases be stored twice — if an HTTP request times out on the device after the proxy has already persisted the sample, and the device then retries the send on the other protocol.

ACKNOWLEDGMENT

This project was carried out for the Internet of Things course (A.Y. 2025-2026) at the University of Bologna, taught by Prof. Marco Di Felice and Prof. Luciano Bononi. We thank them for their guidance throughout the design and testing of this system.

REFERENCES

[1] Espressif Systems, “ESP-IDF Programming Guide,” [Online]. Available: https://docs.espressif.com/projects/esp-idf/

[2] FreeRTOS, “FreeRTOS Reference Manual,” [Online]. Available: https://www.freertos.org/

[3] Z. Shelby, K. Hartke, and C. Bormann, “The Constrained Application Protocol (CoAP),” RFC 7252, IETF, 2014.

[4] OASIS, “MQTT Version 5.0,” OASIS Standard, 2019.

[5] InfluxData, “InfluxDB 2.x Documentation,” [Online]. Available: https://docs.influxdata.com/influxdb/v2/

[6] Grafana Labs, “Grafana Documentation,” [Online]. Available: https://grafana.com/docs/grafana/latest/

[7] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, Time Series Analysis: Forecasting and Control, 5th ed. Hoboken, NJ: Wiley, 2015.

[8] statsmodels developers, “statsmodels.tsa.arima.model.ARIMA,” [Online]. Available: https://www.statsmodels.org/